

A PROJECT REPORT ON

FINSIGHTS

Smart Finance Insights

Milestone 1 & 2 Report

Submitted by

Biswajeet Behera

Under the guidance of

Dr. V. Suresh Kumar

Table of Contents

- 1. Introduction** 1
 - 1.1 Background..... 1
 - 1.2 Problem Statement 1
 - 1.3 Objectives 2
 - 1.4 Scope of this Report 2
- 2. Tools and Technologies Used**..... 3
- 3. System Design** 4
 - 3.1 Architectural Approach 4
 - 3.2 Data Storage..... 4
 - 3.3 Module Overview 5
- 4. Milestone 1: Core Foundation** 6
 - 4.1 User Registration and Login 6
 - 4.2 Dashboard Shell..... 7
 - 4.3 Expense Tracking 8
 - 4.4 Income Tracking 9
- 5. Milestone 2: Extended Features** 10
 - 5.1 Budget Management..... 10
 - 5.2 Goal Planning..... 11
 - 5.3 Investment Portfolio 12
 - 5.4 Financial Health Score 13
 - 5.5 Reports 13
 - 5.6 Notifications, Admin Panel and Profile 14
- 6. Challenges Faced** 15
- 7. Conclusion and Future Scope**..... 15
- References** 16

1. Introduction

1.1 Background

Every semester I've noticed the same thing happen to my own wallet — expenses pile up across a dozen small transactions and by the time I sit down to actually check where the money went, half of it is already a mystery. Most students and young professionals I know track money either in a scattered set of notes-app entries or not at all, and the handful of finance apps available either push you toward a bank integration you don't trust or bury the useful analytics behind a subscription. That gap is what pushed me toward this project: a personal finance manager that runs entirely on the desktop, keeps all data local, and still gives a proper picture of income, spending, goals and investments in one place.

FinSights (branded internally during early prototyping as 'Smart Finance Insights') is the result of that idea — a Python desktop application built with CustomTkinter for the interface. I picked a desktop-first approach deliberately rather than jumping straight to a web app, mainly because it let me focus on getting the core logic and UI right without also having to handle hosting, authentication over a network, or browser quirks.

1.2 Problem Statement

Individuals managing personal finances across income, expenses, budgets, savings goals and investments typically rely on either fragmented tools (a spreadsheet for expenses, a separate app for investments, a mental note for goals) or on finance apps that demand sensitive bank credentials. There is a need for a single, self-contained application that consolidates these categories, gives the user visual feedback on their financial health, and does not require any external account or internet dependency to function.

1.3 Objectives

- Build a desktop application that lets a user record and categorise income and expense transactions.
- Provide budget creation and tracking per category, with visual indicators of overspending.
- Allow users to define and monitor progress toward savings goals.
- Track investment holdings across different instrument types.
- Compute and display an overall financial health score derived from the user's data.
- Generate exportable reports (PDF/Excel) summarising the user's financial activity.
- Keep the codebase modular so that new features can be added without reworking the whole dashboard.

1.4 Scope of this Report

This report documents the work completed under Milestone 1 and Milestone 2 of the project, as guided by Dr. V. Suresh Kumar. Milestone 1 covers the foundational pieces — user registration, authentication, the dashboard shell, and expense/income tracking. Milestone 2 builds on top of that foundation to add budgeting, goal planning, investment tracking, the financial health score, reporting,

notifications, an admin panel, and the user profile section. Work done afterward (a later analytics module and multi-currency support) falls outside Milestone 1 and 2 and is not covered here.

2. Tools and Technologies Used

I kept the tech stack intentionally lightweight so I could iterate quickly without fighting a heavy framework. Everything runs locally with no server component.

Language	Python 3
GUI Framework	CustomTkinter (built on Tkinter)
Supplementary UI	Standard Tkinter ttk widgets (Treeview, styles) for tables
Data Storage	Local JSON files (transactions, budgets, goals, investments, notifications, users)
Charts / Visualisation	Matplotlib, embedded inside the dashboard canvases
Report Export	ReportLab (PDF export), OpenPyXL (Excel export)
Version Control	Git / GitHub

3. System Design

3.1 Architectural Approach

The codebase follows a loosely MVC-inspired structure. The dashboard window (`dashboard.py` and `base_dashboard.py`) acts as the shell — it owns the window, sidebar navigation, theming, and shared helper methods. Each functional area of the app (expenses, income, budget, goals, investments, health, notifications, reports, admin, profile) lives in its own file under the `modules/` folder and is written as a mixin class that gets combined into the final `FinsightsDashboard` class. This kept individual files small and let me work on one feature at a time — say, budgeting — without touching the code for goals or investments at all, which mattered a lot once the project crossed a few thousand lines.

`config.py` is the closest thing to a 'model' layer: it centralises the colour palette, sizing constants, category lists, currency formatting helpers, and — importantly — the load/save functions (`_ld`, `_sv`, `_ldd`) that every module uses to read from and write to the JSON data files. Having one place responsible for persistence meant that whenever I changed how a record was stored, I only had to touch one file instead of hunting through every module.

3.2 Data Storage

Rather than setting up a full database engine for what is, at this stage, a single-user desktop app, I stored data as flat JSON files under a `data/` directory — one file each for transactions, budgets, goals, investments, and notifications, plus a separate users file for registered accounts. Each record carries an id generated at creation time, which is what the edit/delete flows key off. This was simple to implement and easy to inspect/debug during development, at the cost of not scaling well to concurrent access — which is an acceptable trade-off for a personal, single-user tool.

3.3 Module Overview

<i>regform.py</i>	User registration and login screen, password hashing/verification
<i>dashboard.py / base_dashboard.py</i>	Main window shell, sidebar navigation, shared UI setup
<i>modules/overview.py</i>	Landing dashboard view — summary cards, quick stats
<i>modules/expenses.py</i>	Add / edit / delete expense entries
<i>modules/income.py</i>	Add / edit / delete income entries
<i>modules/budget.py</i>	Category-wise budget creation and progress bars
<i>modules/goals.py</i>	Savings goal creation and progress tracking
<i>modules/investments.py</i>	Investment holdings — add / edit / delete
<i>modules/health.py</i>	Financial health score computation and display
<i>modules/reports.py</i>	PDF / Excel report generation, chart rendering
<i>modules/notifications.py</i>	In-app notifications and alerts
<i>modules/admin.py</i>	Admin-only panel — user oversight, broadcast messages
<i>modules/profile.py</i>	User profile, linked accounts, password change, logout

4. Milestone 1: Core Foundation

Milestone 1 was about getting the skeleton of the app right — an account system, a working dashboard window, and the ability to log day-to-day income and expenses. Nothing here is flashy, but it's the part everything else in Milestone 2 depends on, so I spent more time than I expected just making sure the window handling, styling, and data flow were solid before adding more modules on top.

4.1 User Registration and Login

New users sign up through a registration form that collects the basic account details and stores credentials in the local users file. Passwords are never stored in plain text — I used HMAC-based hashing (`hash_password` / `verify_password` in `config.py`) so that even the local JSON file doesn't expose a readable password. Once registered, the same screen doubles as the login form for returning users.

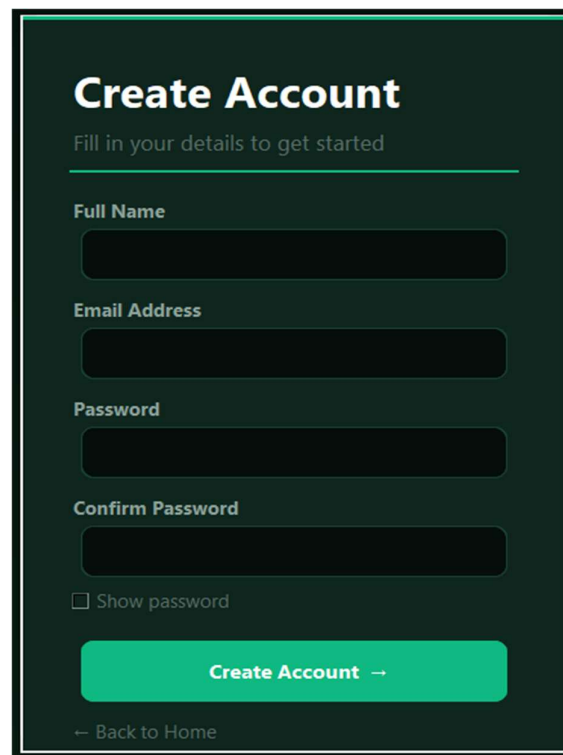
The image shows a user registration screen with a dark green background. At the top, the title "Create Account" is displayed in a large, bold, white font. Below the title, a subtitle "Fill in your details to get started" is shown in a smaller, lighter green font. A horizontal line separates the subtitle from the form fields. The form consists of four input fields, each with a label above it: "Full Name", "Email Address", "Password", and "Confirm Password". The labels are in a bold, white font. The input fields are dark green with rounded corners. Below the "Password" field, there is a checkbox labeled "Show password" in a small, white font. At the bottom of the form, there is a large, rounded rectangular button with a bright green background and the text "Create Account →" in a bold, white font. Below the button, there is a link "← Back to Home" in a small, white font.

Fig 4.1 — User registration screen

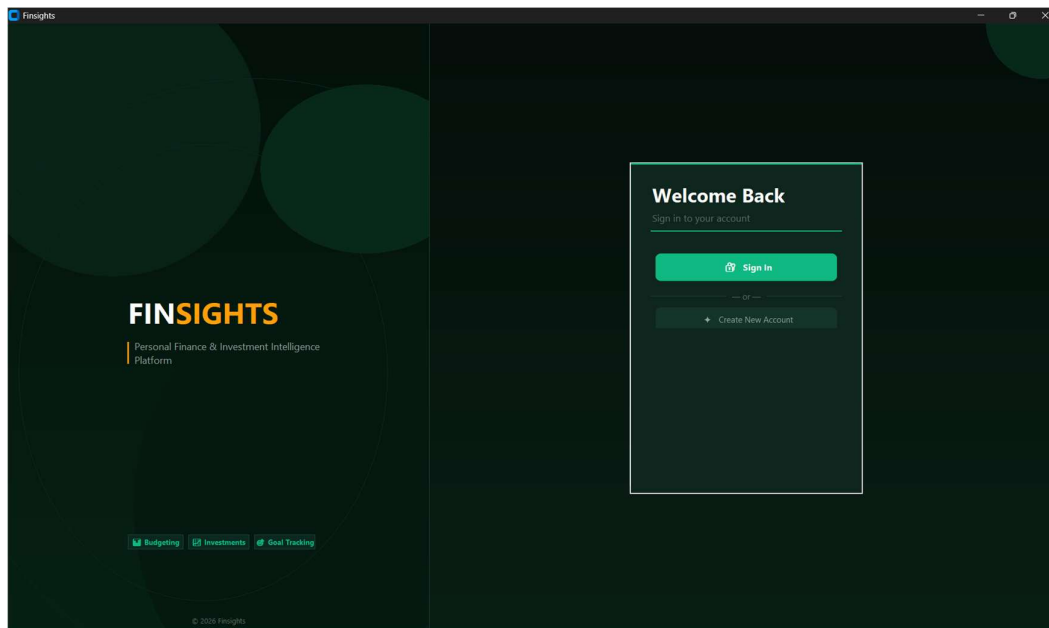


Fig 4.2 — Login screen

4.2 Dashboard Shell

After login, the user lands on the main dashboard window. This is where a lot of the early effort went — the sidebar navigation, the dark fintech-style theme, consistent card styling, scroll handling across different sections, and window resizing all had to be sorted out here before any individual module could be plugged in cleanly. The overview screen itself pulls together summary cards (income, expenses, balance, budget status) so the user gets a snapshot the moment they log in, rather than having to click into each section separately.

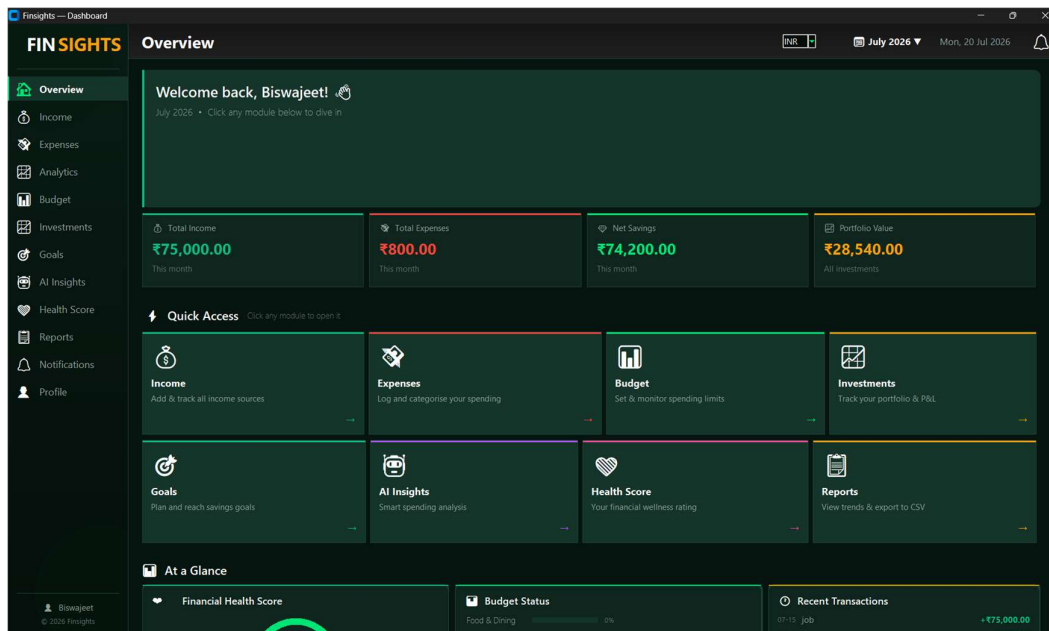


Fig 4.3 — Dashboard overview screen

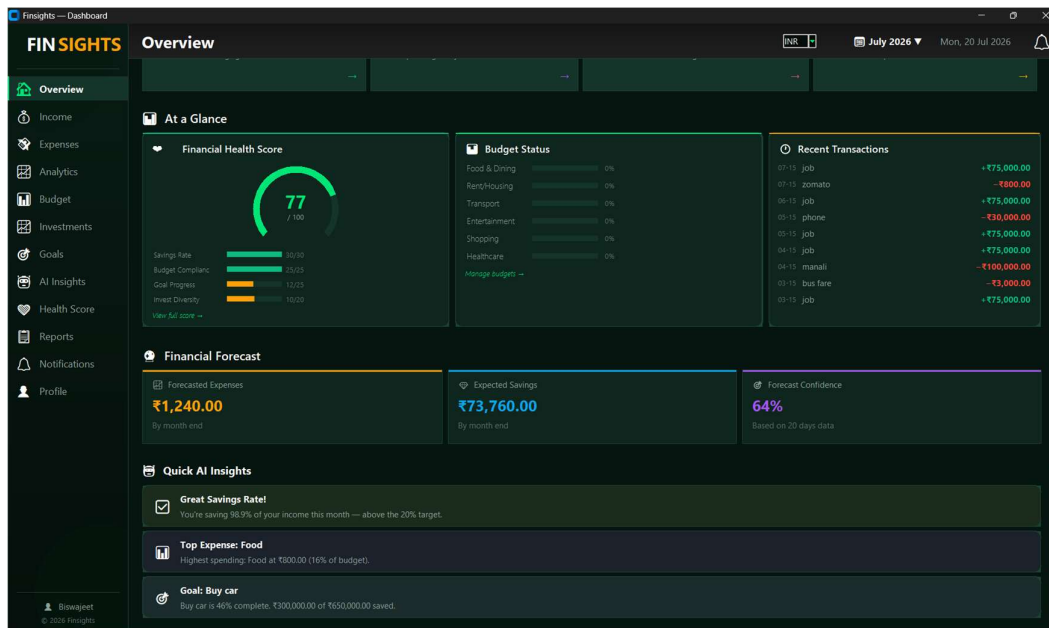


Fig 4.4 — Dashboard overview, alternate view

4.3 Expense Tracking

The expense module lets the user add a transaction with an amount, category, date and optional note, and lists all recorded expenses in a sortable table. Editing and deleting are handled inline rather than through a separate page, which keeps the interaction quick for something users will do often.

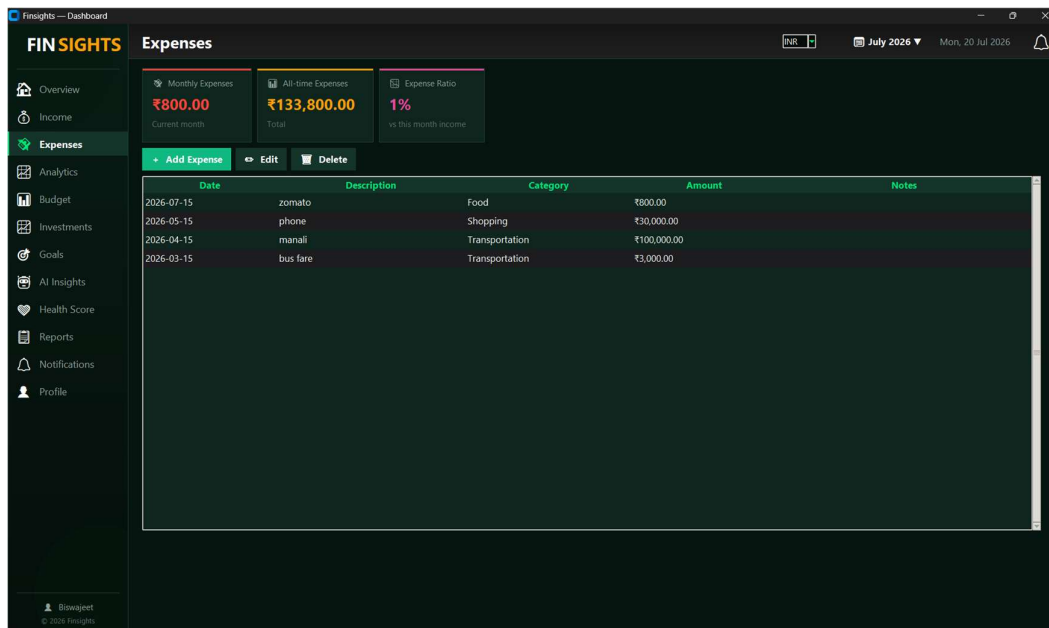


Fig 4.5 — Expense management screen

4.4 Income Tracking

Income entries follow the same pattern as expenses — add, edit, delete — but are kept in a separate category set (salary, freelance, investment returns, etc.) since income sources tend to be fewer and more predictable than expense categories. Keeping income and expenses as parallel-but-separate

modules made the balance calculation on the overview page a straightforward subtraction rather than something that needed a shared transaction-type flag threaded through every screen.

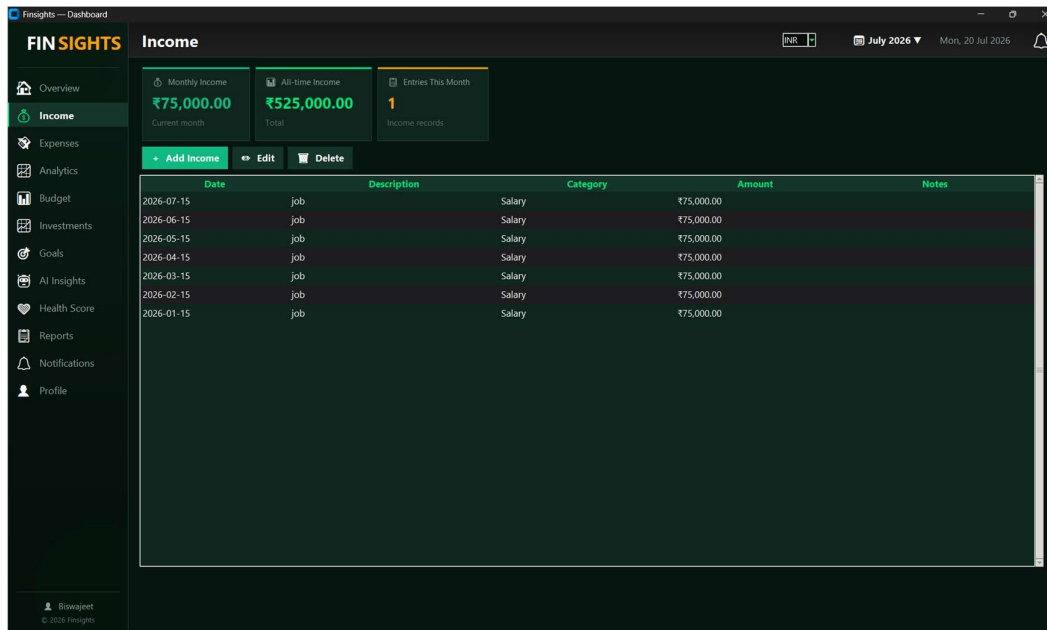


Fig 4.6 — Income management screen

5. Milestone 2: Extended Features

With the core in place, Milestone 2 focused on the features that make the app genuinely useful for planning rather than just record-keeping — budgets, goals, investments, a health score, reporting, and the supporting admin/notification/profile layers.

5.1 Budget Management

Users can set a spending limit per category for the current period. The budget screen compares actual spending (pulled from the expense records) against the set limit and renders a progress bar per category, colour-coded so it's obvious at a glance which categories are running over. Adding or editing a budget opens a small dialog rather than a full page, which matches the pattern used elsewhere in the app for quick data entry.

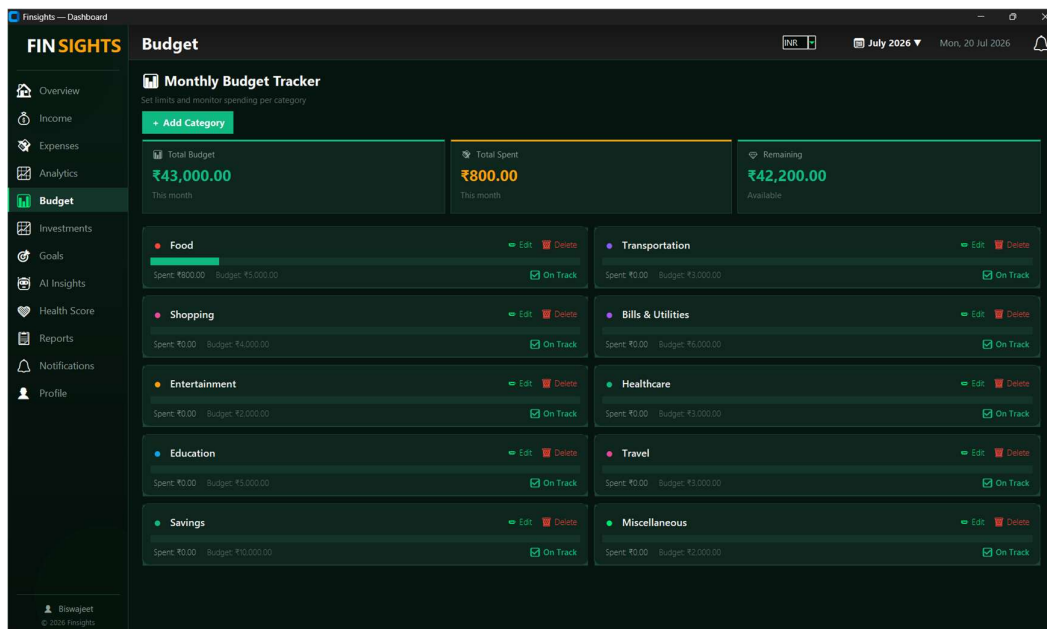


Fig 5.1 — Budget management screen

Add Budget Category

Category Name

Currency: INR

Monthly Budget Limit

Save Cancel

Fig 5.2 — Add / edit budget dialog

5.2 Goal Planning

The goals module lets a user define a savings target — say, an emergency fund or a purchase they're saving toward — along with a target amount and, optionally, a deadline. Progress is tracked against contributions and shown visually, which was one of the more satisfying pieces to build since it's the closest thing in the app to positive reinforcement for the user rather than just a ledger.

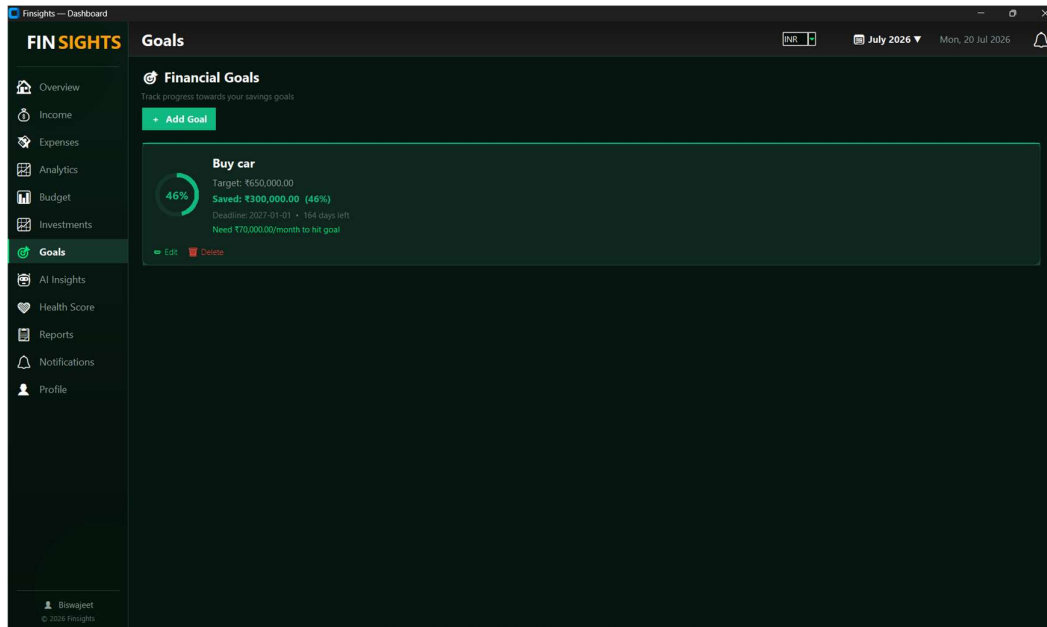


Fig 5.3 — Goal planning screen

The screenshot shows the 'Add New Goal' dialog box. It has a title bar with a feather icon, the text 'Add New Goal', and standard window controls. The dialog contains the following fields: 'Goal Name' (text input), 'Currency' (dropdown menu showing 'INR' with a green checkmark), 'Target Amount' (text input), 'Already Saved' (text input showing '0'), and 'Target Date (YYYY-MM-DD)' (text input). At the bottom are two buttons: 'Save' (green) and 'Cancel' (grey).

Fig 5.4 — Add / edit goal dialog

5.3 Investment Portfolio

This section tracks investment holdings — type of instrument, amount invested, and related details — so a user gets a consolidated view of where their money is deployed outside of day-to-day spending. It doesn't pull live market data (that would need a network dependency I deliberately avoided at this stage); instead, the user enters and updates values manually, which is a reasonable trade-off for a first version.

Asset	Symbol	Type	Qty	Buy Price	Curr Price	Value	P&L	P&L %
Reliance	REL	Stocks	10.0	₹2,500.00	₹2,800.00	₹28,000.00	₹3,000.00	+12.0%
Bitcoin	BTC	Crypto	0.03	₹20,000.00	₹18,000.00	₹540.00	-₹60.00	-10.0%

Fig 5.5 — Investment portfolio screen

Add Investment

Asset Name

Symbol / Ticker

Type: Stocks

Currency: EUR

Quantity

Buy Price

Current Price

Notes

Fig 5.6 — Add / edit investment dialog

5.4 Financial Health Score

One of the features I was most interested in building was a single score that reflects the user's overall financial standing, rather than making them interpret four different screens on their own. The health module pulls together signals from income vs. expenses, budget adherence, and goal progress into one score, displayed with a breakdown so the user can see which factor is dragging the number down instead of just being handed an opaque number.

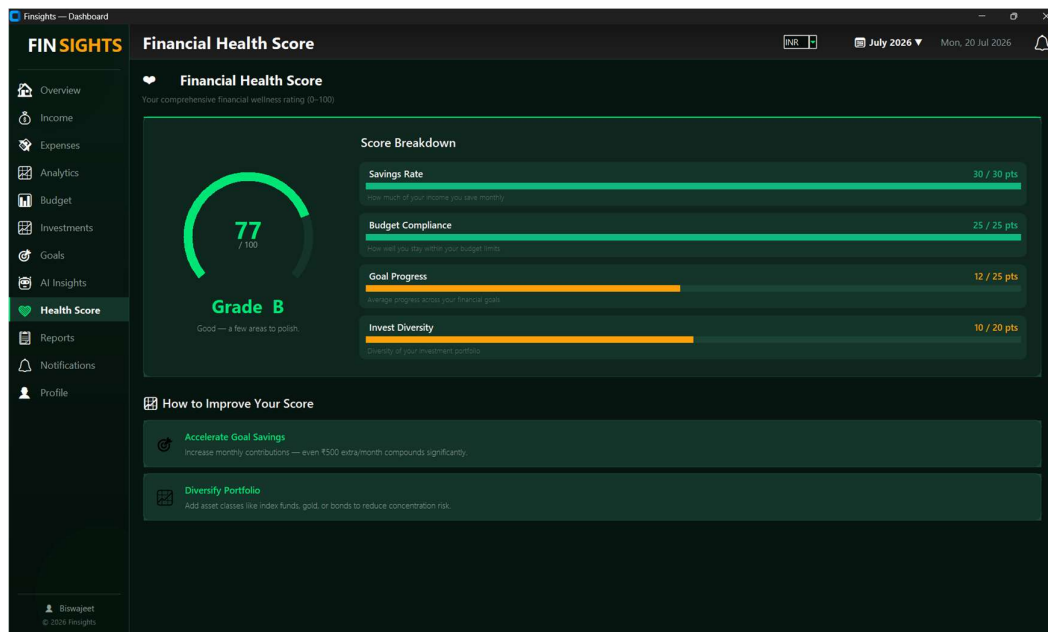


Fig 5.7 — Financial health score screen

5.5 Reports

The reports module lets users export a summary of their financial activity as a PDF (via ReportLab) or an Excel workbook (via OpenPyXL), with charts generated through Matplotlib and embedded into the export. This was one of the fiddlier parts of Milestone 2 — getting chart images to render correctly inside a generated PDF took a few passes, mainly around sizing and making sure the export path resolved correctly across different data states (e.g. a user with no transactions yet).

5.6 Notifications, Admin Panel and Profile

Notifications surface budget warnings and goal milestones inside the app itself. The admin panel is a separate view intended for oversight — reviewing registered users and sending broadcast messages — and is only reachable by an admin account. The profile section rounds out Milestone 2, letting a user edit their details, link additional accounts, change their password, and log out or delete their account.

6. Challenges Faced

- CustomTkinter doesn't expose every styling hook that plain Tkinter/ttk does, so a few widgets (like the Treeview tables) had to be styled through `ttk.Style()` separately, which meant keeping two theming systems visually consistent by hand.
- Getting the mouse-wheel scroll binding to work reliably across nested scrollable frames took a few iterations — an earlier version leaked bindings and caused the scroll to lock up after switching sections a few times.
- Deciding on a JSON-file storage approach instead of a proper database was a deliberate simplification for this stage, but it meant writing my own small set of load/save/id-generation helpers rather than relying on an ORM.
- Balancing the financial health score formula so it felt meaningful rather than arbitrary needed a couple of rounds of testing with sample data before the numbers looked reasonable.
- Embedding Matplotlib charts inside both the CustomTkinter dashboard and the exported PDF/Excel reports required two different rendering paths, which took some trial and error to keep visually consistent.

7. Conclusion and Future Scope

By the end of Milestone 2, FinSights covers the full loop of a basic personal finance workflow: registering an account, logging income and expenses, setting budgets, planning goals, tracking investments, seeing a consolidated health score, and exporting reports — all running locally without any external dependency. Building it module-by-module made it easier to keep scope under control between sessions, which mattered given this was developed incrementally rather than in one continuous sprint.

Beyond this report's scope, work has already begun on an analytics module and multi-currency support, and there's room to extend the app further with recurring-transaction automation, data backup/import, and possibly a lightweight database backend if the JSON approach starts to show its limits with larger datasets. For now, though, Milestone 1 and 2 together form a working, self-contained application that does what it set out to do.

References

- CustomTkinter Documentation — <https://customtkinter.tomschimansky.com/>
- Python Software Foundation — Tkinter Documentation — <https://docs.python.org/3/library/tkinter.html>
- Matplotlib Documentation — <https://matplotlib.org/stable/>
- ReportLab User Guide — <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- OpenPyXL Documentation — <https://openpyxl.readthedocs.io/>